Framework-ul tău de colecții

Teorie · noduri, contract și implementare, tablou dinamic, coadă circulară, hash, `yield`, fail-fast

1. Lista înlănțuită: nodul și legătura

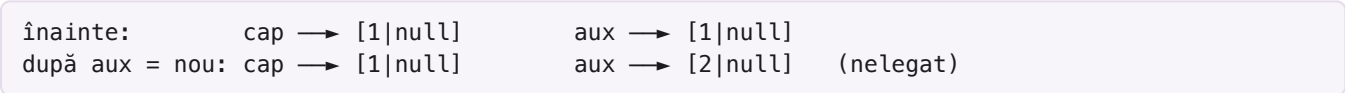
Fiecare element stă în propriul obiect **nod**: valoarea și o referință spre nodul următor. Lista ține primul nod (`cap`) și ultimul (`coada`). Ultimul nod are `Urmator == null`.



Cinci lucruri decid dacă o listă înlănțuită chiar funcționează.

a) Legătura e un câmp al nodului, nu o variabilă

Când parcurgi lanțul cu o variabilă locală (`aux = cap`, apoi `aux = aux.Urmator`), `aux` ține o **copie a referinței**: adresa unui nod, nu nodul. `aux = nodNou` schimbă doar copia. `aux` arată acum spre nodul nou, dar ultimul nod din lanț are tot `Urmator == null`. Lanțul e neschimbat, iar nodul nou nu e legat de nimic.



Rulat: o listă cu un nod are tot **un** nod după o astfel de adăugare. Lanțul se lungește doar când scrii în **câmpul** `Urmator` al ultimului nod.

b) `readonly` îngheață lanțul

Un câmp `readonly` se poate scrie doar în constructor. Dacă `Urmator` e `readonly`, după ce nodul a fost creat nimic nu mai poate fi legat după el, iar compilatorul refuză orice încercare (`CS0191`). Lista nu mai poate crește și nu se poate scoate nimic din mijloc. La fel un `cap readonly`: nu mai poți adăuga la început și nu poți șterge primul element. Legăturile sunt tocmai partea care se schimbă.

c) Nodul e un detaliu de implementare

Cine folosește lista vrea să pună un `string`, nu să construiască noduri. Dacă metodele primesc un nod, apelantul poate trimite unul care are deja un `Urmator` (și aduce cu el un lanț întreg) sau unul care e deja în altă listă. Atunci lista nu mai controlează ce e în ea. Nodul devine `private class Nod`, **în interiorul** listei: nu se vede din afară, iar metodele primesc și întorc `T`. Mai e un motiv: `ListaTablou` n-are noduri. O interfață care ar vorbi despre noduri n-ar putea fi implementată pe un tablou.

d) Lista goală e o stare normală

O listă nouă n-are niciun element: `cap == null`, `coada == null`, `Count == 0`. Un constructor care cere un prim nod face lista goală imposibilă, iar o metodă care începe cu `cap.Urmator` aruncă `NullReferenceException` fix pe lista goală. Două câmpuri în plus fac lista ieftină: `count`, ținut la zi la fiecare modificare, dă `Count` fără parcurgere; `coada` îi permite lui `Adauga` să lege direct după ultimul nod, O(1). Fără ea, fiecare `Adauga` parcurge tot lanțul, iar n adăugări costă cam n²/2 pași.

e) Capătul lanțului e `null`, deci tipul e `Nod?`

Proiectul are `<Nullable>enable</Nullable>`. Acolo `Nod` înseamnă „niciodată null”, iar `Nod?` înseamnă „poate fi null”. Ultimul nod are `Urmator == null` și lista goală are `cap == null`, deci câmpurile sunt `Nod?`. Cu `Nod` simplu, fiecare `null` scris acolo dă `warning CS8625: Cannot convert null literal to non-nullable reference type.` Avertismentul are dreptate: tipul promite ceva ce codul nu ține.

Restul operațiilor. `this[i]`: nu poți sări la poziția `i`, pleci din `cap` și urmezi `Urmator` de `i` ori, deci O(n). Ștergerea unui nod: nodul de dinainte își schimbă `Urmator` peste el. Dacă ai șters ultimul nod, `coada` se mută

pe cel dinainte. Altfel rămâne pe un nod scos din lanț.

2. Ce e un framework de colecții

Un **contract** (interfața) spune *ce* poate face o colecție. O **implementare** (clasa) spune *cum*. Codul scris pe contract merge pe orice implementare: o funcție care primește `ILista<string>` nu știe și nu-i pasă dacă în spate e un tablou sau un lanț de noduri. E același lucru pe care l-ai făcut în academy cu `Repository<T>` și `ITextMapper<T>`. `System.Collections.Generic` e exact asta: `IEnumerable<T>`, `ICollection<T>`, `IList<T>`, `IDictionary<K, V>` deasupra, `List<T>`, `Dictionary<K, V>`… dedesubt. În etapa asta îl scrii pe al tău:

```
IEnumerable<T>                       (din .NET: ce face foreach posibil)
 └─ IColectie<T>                     Count, Adauga, Sterge, Contine,
     │                               Goleste, CaTablou
     ├─ ILista<T>                    this[i], Insereaza, StergeLa, IndexOf
     ├─ IStiva<T>                    Pune, Scoate, Varf
     ├─ ICoada<T>                    Intra, Iese, Primul
     └─ IMultime<T>                  Reuniune, Intersectie
IDictionar<K, V>                     this[k], Adauga, IncearcaSaIa, ...

ColectieAbstracta<T> : IColectie<T>  Contine, CaTablou, ToString
 ├─ ListaTablou<T>       : ILista<T>      tablou + dublare
 ├─ ListaInlantuita<T>   : ILista<T>      noduri
 ├─ StivaTablou<T>       : IStiva<T>      tablou
 ├─ CoadaCirculara<T>    : ICoada<T>      tablou circular
 └─ MultimeHash<T>       : IMultime<T>    are un DictionarHash<T, bool>
DictionarHash<K, V>      : IDictionar<K, V>  găleți + lanțuri
```

Clasa abstractă ține ce se poate scrie o singură dată pentru toate. `Contine` e doar „parcurg cu `foreach` până găsesc”, `CaTablou` e „parcurg și copiez”. Amândouă au nevoie doar de `GetEnumerator`, pe care fiecare clasă îl scrie altfel. Asta e **Template Method**: pașii mari stau în bază, pasul care diferă e abstract. `Contine` e `virtual` pentru că `MultimeHash` are o cale mult mai scurtă decât parcurgerea (§5).

Compoziție, nu moștenire: o mulțime *nu este* un dicționar, dar își poate ține elementele ca chei într-unul. `MultimeHash` are un câmp `DictionarHash<T, bool>` și îi deleagă munca.

3. Tabloul dinamic: `Count`, `Capacitate`, dublarea

Un tablou are lungime fixă. O listă pe tablou ține un tablou **mai mare** decât are nevoie și un `count`: câte locuri sunt ocupate. `Capacitate` = lungimea tabloului, `Count` = câte elemente ai pus. Cât timp `Count < Capacitate`, `Adauga` scrie pe poziția `count` și atât.

Când se umple: alocă un tablou **de 2 ori** mai mare, copiază tot, aruncă tabloul vechi. Așa face și `List<T>`: `Capacity` trece prin 0, 4, 8, 16… În academy ai scris `studentEnrolments.Capacity = Items.Count` (`EnrolmentRepository`): îi spui listei de la început câte locuri să rezerve, deci nu mai dublează și nu mai copiază pe parcurs.

De ce e rapid în medie. Pentru 100.000 de `Adauga` pornind de la 4 locuri, copierile se fac la 4, 8, 16, …, 65.536 de elemente: în total **131.068** de copieri, adică sub 2 pe element. Fiecare dublare e scumpă, dar e plătită de toate adăugările ieftine dinaintea ei. Se spune că `Adauga` e **O(1) amortizat**.

> **Creșterea cu +1.** În `ex7/Canal.cs`, `Aboneaza` face un tablou cu **un** loc în plus și copiază tot, la fiecare apel. Pentru 100.000 de abonați: 1 + 2 + … + 99.999 = **4.999.950.000** de copieri. De 38.000 de ori mai mult, pentru același rezultat.

`Insereaza(i, x)` și `StergeLa(i)` mută tot ce e după `i` cu o poziție. Pe poziția 0 asta înseamnă toată lista: O(n).

4. Stiva și coada circulară

Stiva pe tablou: vârful e ultimul element ocupat. `Pune` și `Scoate` lucrează doar la capătul ăsta, deci O(1).

Coada scoate din față și adaugă în spate. Varianta naivă scoate de pe poziția 0 și mută toate celelalte elemente cu un loc la stânga: fiecare `Iese` costă O(n). Varianta circulară nu mută nimic. Ține `primul` (unde e primul element) și `count`, iar elementul al `i`-lea stă pe poziția `(primul + i) % Length`. Când ajunge la capătul tabloului, continuă de la 0.

Starea din Pas 4, după ce 10, 20, 30, 40 au intrat, 10 și 20 au ieșit, iar 50 și 60 au intrat:

```
poziția:     0    1    2    3
elemente: [ 50 | 60 | 30 | 40 ]      primul = 2, count = 4
                      ▲
                 primul element
coada:    30, 40, 50, 60
```

Tabloul e plin abia când `count == Length`. Atunci dublezi și copiezi **în ordinea din coadă** (30, 40, 50, 60 de la poziția 0), nu în ordinea din tablou.

5. Hash: găleți, `GetHashCode`, `Equals`

Dicționarul are un tablou de **găleți**. Fiecare găleată e începutul unui lanț de noduri `(cheie, valoare)`. Căutarea unei chei are doi pași:

1. `GetHashCode(cheie)` dă un `int`; din el iese găleata: `(hash & 0x7FFFFFFF) % galeti.Length`. `& 0x7FFFFFFF` șterge semnul, fiindcă un hash poate fi negativ, iar în C# `-7 % 8` e `-7`, nu un index;
2. în lanțul acelei găleți, compari fiecare cheie cu `Equals`.

Două chei diferite pot ajunge în aceeași găleată: e o **coliziune**. De aceea găleata e un lanț, nu un singur loc. Cât timp lanțurile sunt scurte, căutarea nu depinde de câte chei sunt: O(1) în medie.

Factorul de încărcare = `Count / NumarGaleti`. Când ar trece de 0,75, dublezi gălețile. Cum găleata se calculează cu `% galeti.Length`, după dublare fiecare cheie își recalculează locul: nu copiezi tabloul, redistribui nodurile. În Pas 5, a 7-a cheie trece de 8 × 0,75 = 6, deci gălețile devin 16.

> **Contractul:** dacă `a.Equals(b)`, atunci `a.GetHashCode() == b.GetHashCode()`. Pentru o clasă scrisă de tine, implicit, `Equals` compară **referințele**, iar `GetHashCode` dă un număr per obiect. Două `new Pozitie(8, 5)` au hash-uri diferite: ajung în găleți diferite și nu se compară niciodată. Valoarea pusă cu o cheie nu mai poate fi găsită decât cu **același obiect**. Dacă suprascrii doar `Equals`, compilatorul te avertizează (`CS0659`): fără `GetHashCode`, cheile egale tot în găleți diferite ajung.

`EqualityComparer<T>.Default` e calea prin care codul generic cheamă `Equals` și `GetHashCode` pe un `T` oarecare. `==` nu se compilează pe un `T` fără constrângeri.

Ordinea nu e garantată. Parcurgerea merge găleată după găleată, deci depinde de hash-uri. La `string`, .NET calculează hash-ul diferit la **fiecare pornire** a programului: aceeași cheie `"Ana"` are alt hash la rularea următoare. De aceea ieșirile din fișă trec prin `Sortat`.

6. `IEnumerable<T>` și `yield return`

`foreach` nu știe nimic despre liste. Compilatorul îl transformă în apeluri pe enumerator:

```
IEnumerator<string> e = lista.GetEnumerator();
while (e.MoveNext())
{
    string n = e.Current;
    Console.Write(n);
}
```

Deci orice clasă cu `GetEnumerator()` merge în `foreach`, în `string.Join` și în tot ce primește `IEnumerable<T>`. O metodă cu `yield return` e scrisă ca o buclă obișnuită, dar compilatorul o transformă într-un enumerator: fiecare `MoveNext()` rulează codul până la următorul `yield return`, **se oprește acolo** și dă valoarea în `Current`. Următorul `MoveNext()` continuă de pe linia de după `yield return`.

`IEnumerable<T>` moștenește vechiul `IEnumerable` ne-generic, deci clasa trebuie să aibă și un `GetEnumerator()` ne-generic. Se scrie o singură dată, în `ColectieAbstracta`, și îl cheamă pe cel generic.

7. Fail-fast

Dacă în timpul unui `foreach` scoți un element, pozițiile de după se mută și parcurgerea sare peste unul, fără nicio eroare. .NET preferă să **oprească** programul imediat: `InvalidOperationException: Collection was modified`. Mecanismul e un contor:

- colecția are `int versiune`; orice modificare reușită face `versiune++`;
- enumeratorul reține versiunea când începe; de fiecare dată când `foreach`-ul revine în el, după `yield return`, compară. Diferită → aruncă.

Verificarea stă **după** `yield return`, fiindcă acolo se întoarce controlul după ce corpul `foreach`-ului a rulat. Dacă ar sta înaintea lui, ștergerea ultimului element ar trece neobservată: bucla s-ar opri singură, cu lista schimbată.

Pentru ștergeri în timpul parcurgerii folosești `for` descrescător, pe index, ca în Pas 6.

8. Complexități

	Adaugă la final	Inserează la început	`this[i]`	Caută / `Contine`	Scoate
`ListaTablou`	O(1) amortizat	O(n)	O(1)	O(n)	O(n) (`StergeLa`)
`ListaInlantuita`	O(1)	O(1)	O(n)	O(n)	O(n), O(1) la `cap`
`StivaTablou`	O(1) amortizat	—	—	O(n)	O(1) `Scoate`
`CoadaCirculara`	O(1) amortizat	—	—	O(n)	O(1) `Iese`
`DictionarHash`, `MultimeHash`	O(1) mediu	—	—	O(1) mediu	O(1) mediu

Ce înseamnă în cifre: 1000 de căutări într-o `ListaTablou` de 100.000 de numere iau **sute de milisecunde**; aceleași 1000 de căutări în `DictionarHash` iau **sutimi de milisecundă**. Diferența crește odată cu numărul de elemente.

Etapa 2. Tot ce scrii aici există deja în .NET, gata testat:

Al tău	Din .NET
`ListaTablou<T>`	`List<T>`
`ListaInlantuita<T>`	`LinkedList<T>` (dublu înlănțuită, fără index)
`StivaTablou<T>`, `CoadaCirculara<T>`	`Stack<T>`, `Queue<T>`
`DictionarHash<K, V>`, `MultimeHash<T>`	`Dictionary<K, V>`, `HashSet<T>`